JAC: A PRACTICAL PATH FROM A PHYSICS QUESTION TO A TRUSTED NUMBER

21-Oct-2026

This note is written for an occasional user: someone who knows enough Julia to start a REPL, has JAC installed, and has a real atomic-physics problem -- but has not used JAC before and does not want to read the whole source tree to get a first trustworthy number out of it.

Ten steps answer three questions: (A) does JAC fit the problem at all, (B) how is that problem turned into proper JAC input, and (C) how is the result run and judged before it is quoted to anyone.

PART 1 -- THE TEN STEPS, IN BRIEF

A. DOES JAC FIT THE PROBLEM?

1. Classify the problem. JAC distinguishes seven kinds of computation: structure/properties, representations (RAS), interactive use, cascades, empirical models, plasma, and symbolic Racah algebra. If the problem does not land in one of these, JAC is probably the wrong tool today -- say so before writing any input.

2. Search for the SAME PHYSICS among the examples, not the same function name. Look in examples/examples.jl and the example-Xx.jl branch files for the process or property in question. A branch carrying a dated "Last successful" line, with its own numbers written on that line, is worth more than any manual: a human has already judged that physics.

3. Check the size and accuracy ceiling before committing to the case. How many open shells, how many configuration state functions (CSF) this implies, whether the radial box will need to be matched to the orbitals rather than left at a default, whether a RAS treatment is even feasible at this nuclear charge and shell count. This turns "can JAC do this in principle" into "can JAC do THIS case on THIS machine" -- a more honest and more useful question.

B. FORMULATE THE INPUT

4. Pick the right entry point for the problem's SHAPE. One standard property on one configuration calls for the simple ForPedestrians functions; a general structure, property or process computation goes through Atomic.Computation and perform(); several charge states tied together by processes call for Cascade.Computation. Starting from the wrong entry point is the most common reason a first attempt feels harder than it needs to be.

5. Read the Settings docstring for the chosen process or property BEFORE writing any input, with `? Module.Settings` at the REPL. Every knob has a name, a default and a one-line explanation right there; this is JAC's whole input philosophy, and it removes the need to guess a

field name from memory.

6. Decide the mean field and the correlation model as a PHYSICS CHOICE, not as a default left untouched. The self-consistent-field route (Dirac-Fock-Slater, average-level, extended optimized-level), which correlation layers are included, which orbitals are frozen, and which interaction (Coulomb, or Coulomb plus Breit) is used -- these five choices can move a computed number by tens of per cent even when nothing else in the input changes.

C. RUN IT AND JUDGE THE RESULT

7. Run the smallest version FIRST: one reference configuration, no correlation layer, printout=true. This is cheap, and it is where a wrong shell, a mismatched grid or a non-converging field shows up in seconds rather than after a long correlation study.

8. Scale up ONE axis at a time -- one more correlation layer, a larger box, one more reference configuration per run -- and read the INCREMENT each step produces, not only the running total. Two calculations can agree on a total correlation energy while disagreeing badly about where it came from, and only the sequence of increments says whether the next layer is worth adding.

9. Read JAC's OWN built-in verdicts before reading the number itself: Cowan's cancellation factor, the Babushkin/Coulomb gauge-agreement ratio, and any printed convergence warning. These are computed and displayed specifically so that a user need not independently judge whether a given number can be trusted.

10. Compare the result against something JAC did NOT compute: a published value, a known limiting case, a sum rule, or an independent method already validated inside JAC. Gauge agreement and a clean convergence flag prove internal CONSISTENCY; they say nothing about whether the physical model behind the number (field, correlation layer, box) was adequate for the question being asked.

In one sentence: JAC already tells a user most of what is needed to judge a result -- gauge ratios, cancellation factors, dated examples, convergence flags -- and the work above is mostly about making sure those signals are actually read, rather than only the headline number.

==========

PART 2 -- THE SAME TEN STEPS, EXPLAINED FOR AN OCCASIONAL USER

==========

The paragraphs below assume comfort with ordinary Julia syntax (function calls, keyword arguments, arrays) but NO prior exposure to JAC's own data types, module names or conventions. Each step repeats the short form above, then explains it in more detail, usually with a concrete REPL command.

STEP 1 -- Classify the problem against JAC's seven kinds of computation

---

Before touching Julia, state the physics question in one sentence and check which of these seven baskets it falls into:

   (a) level energies or a single atomic PROPERTY of one ion (hyperfine structure, Lande factor, isotope shift, a polarizability, ...);
   (b) a systematically-enlarged REPRESENTATION of the wave function (restricted-active-space, RAS, correlation studies);
   (c) ad-hoc INTERACTIVE use of JAC's data types at the REPL, for a calculation that does not fit a ready-made recipe;
   (d) an atomic CASCADE across three or more charge states, connected by named processes (photoionization, Auger decay, recombination, ...);
   (e) an EMPIRICAL estimate, where no ab-initio treatment is practical (electron-impact ionization, stopping power, tunnel ionization);
   (f) a PLASMA quantity (line shift, ionization balance, average-atom opacity);
   (g) a purely SYMBOLIC simplification of an expression from Racah's algebra (sums of Wigner n-j symbols).

If the question does not obviously belong to one of these, that is useful information on its own: it means either that some translation is needed first (for example, turning an experimental observable into one of the quantities above), or that this really is new territory for JAC, worth flagging rather than forcing into the nearest-looking basket.

STEP 2 -- Look for the same PHYSICS among the examples, not the same name
---

JAC's own convention is to call a QUALIFIED name, such as PhotoEmission.Settings or MultipolePolarizibility.computeAmplitudes, almost never a bare exported function. Searching the source tree for a half-remembered bare name therefore tends to find almost nothing. Searching for the PHYSICS works far better. From a shell, inside the JAC directory:

```
grep -il "polarizability" examples/examples.jl examples/example-*.jl
```

This finds the example files that already exercise the relevant physics. Open the matching example-Xx.jl file and look for a line such as

```
# Last successful:  15-Oct-2026   ## ... quotes its own numbers here
```

A dated "Last successful" line, with numbers written directly on it, means a human already ran this exact kind of case and judged the physics to be right -- not merely that the code did not crash. That example is then the best possible starting template: copy its structure, not just its idea.

If nothing resembling the physics in question appears in any example, it is honest to say so plainly: the capability may exist in src/ but has never been exercised end to end, which is a different (and riskier) situation from reusing a dated, trusted recipe.

STEP 3 -- Check the size and accuracy ceiling before committing
---

Three quick, cheap questions, before any real run is attempted:

- How many OPEN SHELLS does the configuration of interest have, and roughly how many configuration-state functions (CSF) does that imply? A closed-shell or near-closed-shell ion is a very different proposal from a half-filled d- or f-shell, cost-wise.

- Will the problem need a RADIAL BOX matched to the orbitals, rather than a generic default? A box far too small OR far too large both corrupt the basis -- too small truncates outer orbitals silently (a wrong state, not merely an inaccurate one); too large spends a fixed number of splines on empty space. Near-threshold continuum computations in JAC will outright refuse ("enlarge box-size") when the box is clearly unusable, which is the loud and easy failure; a too-small box that still "runs" is the quiet and dangerous one.

- Is a RAS (restricted-active-space) treatment realistic at this nuclear charge? Cost grows quickly with both the number of CSF and the number of open subshells, and there is no simple one-size-fits-all formula -- a short trial at one or two correlation layers, before committing to a deep study, is the practical answer.

The point of this step is to turn a vague "could JAC handle this" into a concrete expectation of running time and basis size, BEFORE any input is written in earnest.

STEP 4 -- Pick the entry point that matches the problem's shape
---

JAC offers several "doors in", and picking the wrong one is the single most common reason a first attempt feels unnecessarily hard:

- For ONE standard property of ONE ion (hyperfine constants, a Lande g-factor, an isotope shift, ...), the ForPedestrians convenience functions in module-ForPedestrians.jl give the shortest path; they wrap the general machinery below and print a readable report.

- For a GENERAL structure, property or process computation, build an Atomic.Computation and hand it to perform():

```
using JenaAtomicCalculator
comp = Atomic.Computation(Atomic.Computation();
          name           = "He-like Fe: 1s 2p -> 1s^2",
          grid           = Radial.Grid(true),
          nuclearModel   = Nuclear.Model(26.),
          initialConfigs = [Configuration("1s 2p")],
          finalConfigs   = [Configuration("1s^2")],
          processSettings = PhotoEmission.Settings(
              PhotoEmission.Settings();
```

```
                        multipoles = [E1], gauges = [UseCoulomb, UseBabushkin]))
        perform(comp)
```

      This single pattern -- build a Computation, call perform() -- is the
      STANDARD route through JAC and should be preferred over calling an
      internal Module.compute...() function directly, even where that
      looks like a shortcut: perform() is where the printed quality
      indicators (Step 9) are produced.

   - For a CASCADE across several charge states, the analogous object is
     Cascade.Computation, built the same way but naming a cascade
     "approach" and the processes to be followed between charge states.

If no Settings-based route exists at all for the quantity in question, that is itself a finding worth recording -- it means the "standard" perform() path has a genuine gap for this corner of the code, not that the user did something wrong.

STEP 5 -- Read the Settings docstring before writing the input

---

Every process or property module in JAC defines its own Settings struct, and that struct's docstring is the authoritative description of every knob available for that computation. At the Julia REPL:

```
   ? PhotoEmission.Settings
```

This prints, for every field: its name, its type, its default value, and a short description of what it controls. Reading this BEFORE writing input avoids two common mistakes: guessing a field name that does not exist (which Julia will reject outright), and silently relying on a default that turns out to matter a great deal for the question being asked (see Step 6).

The same "?" works for any JAC data type as well -- Shell, Subshell, Configuration, Level, LevelSelection and the rest -- and is the fastest way to recover a forgotten constructor without searching the source tree.

STEP 6 -- Decide the mean field and correlation model as a physics choice

---

This is the step most easily skipped by an occasional user, and the one most likely to make a number wrong by a large factor if it is skipped.

JAC's AsfSettings() defaults to a Dirac-Fock-Slater (DFS) mean field, which is convenient and fast but not always the most ACCURATE choice. As one concrete, measured illustration: a static polarizability computed on the same orbitals, same basis, same code, differed by roughly +12% (DFS), +39% (a cruder Hartree-Slater field) and +2.2% (average-level, AL) from a published reference value -- purely because of which mean field was used to generate the orbitals. The physics "engine" computing the response can be essentially exact, while the ACCURACY of the final number is set almost

entirely by this one choice.

Before running anything beyond the smallest test case (Step 7), decide explicitly:

- the mean field: Basics.DFSField(), Basics.ALField(), or Basics.EOLField() (passed as scField = ... in AsfSettings);
- which correlation layers, if any, are to be added on top of the reference configuration, and in what order;
- which orbitals, if any, are held FROZEN while others are allowed to relax;
- whether the Breit interaction is needed in addition to the Coulomb interaction (relevant mainly at higher nuclear charge or for close level crossings).

None of this can be fixed after the fact by looking harder at the output; it has to be decided before the run.

STEP 7 -- Run the smallest possible version first

Before any correlation layer, any box-size study, or any large CSF expansion: run the single reference configuration alone, with

```
perform(comp; printout = true)
```

printout = true makes JAC show its working -- the orbitals it generated, the self-consistent-field iteration, the resulting level energies -- in full. This is the cheapest possible run, often seconds, and it is where a wrong subshell occupation, an obviously mismatched radial grid, or a self-consistent field that fails to converge becomes visible immediately, rather than after an hour spent on a correlation study built on top of a broken foundation.

STEP 8 -- Scale up one axis at a time, and read the increments

---

Once the smallest version runs cleanly, enlarge the calculation ONE dimension per run: one additional correlation layer, OR a larger radial box, OR one more reference configuration -- never several of these at once, because then a change in the result cannot be attributed to any one cause.

For a correlation study in particular, record the ENERGY INCREMENT that each added layer produces, not only the running total. Two different correlation treatments can arrive at nearly the same total correlation energy while disagreeing substantially about where that energy comes from layer by layer -- and it is the shape of that increment sequence, not the final total, that tells a user whether one more layer is still worth the cost or whether the result has effectively converged.

STEP 9 -- Read JAC's own built-in verdicts before reading the number

---

JAC prints two checks alongside many of its results that exist specifically to save the user from having to independently judge trustworthiness:

- Cowan's CANCELLATION FACTOR, which flags a transition amplitude that is the result of a delicate cancellation between large terms -- such an amplitude is numerically fragile even when the arithmetic is correct, and the printed factor is a direct warning of this;

- the BABUSHKIN / COULOMB GAUGE ratio, which compares a transition rate computed in two different, formally equivalent gauges. Close agreement between the two is a necessary (not sufficient) condition for having used a good enough wave function; a ratio far from 1 means the calculation is not yet good enough to trust, regardless of how plausible the number looks.

A completed run that prints a convergence warning, a poor cancellation factor, or a gauge ratio far from unity has NOT passed, even though it finished without crashing. Reading these flags is not optional diligence on top of "the real result" -- they are part of the result.

STEP 10 -- Compare against something JAC did not itself compute

---

Finally, before quoting any number to a colleague or a paper, compare it against something OUTSIDE the calculation just performed: a published experimental or theoretical value, a known exact limiting case (such as a sum rule, a threshold behaviour, or a hydrogenic limit at low nuclear charge), or an independent method already validated inside JAC for a related quantity.

This matters because gauge agreement and a clean convergence flag (Step 9) prove only that the calculation is INTERNALLY CONSISTENT -- that the two gauges agree, that adding one more basis function changed nothing. They say nothing about whether the physical MODEL behind the number (the mean field, the correlation layers included, the size of the radial box) was actually adequate for the question being asked. Only a comparison against an independent anchor can show that.

CLOSING NOTE

None of the ten steps above requires reading JAC's source code. They describe a DISCIPLINE for using a code that is, by design, very permissive about what input it will accept -- and the discipline is what keeps that flexibility from becoming a way to quietly compute a number that happens to run but should not yet be trusted.